

Detector de fraude con tarjeta de crédito: pipeline MLOps reproducible con DVC, MLflow y despliegue en Cloudflare

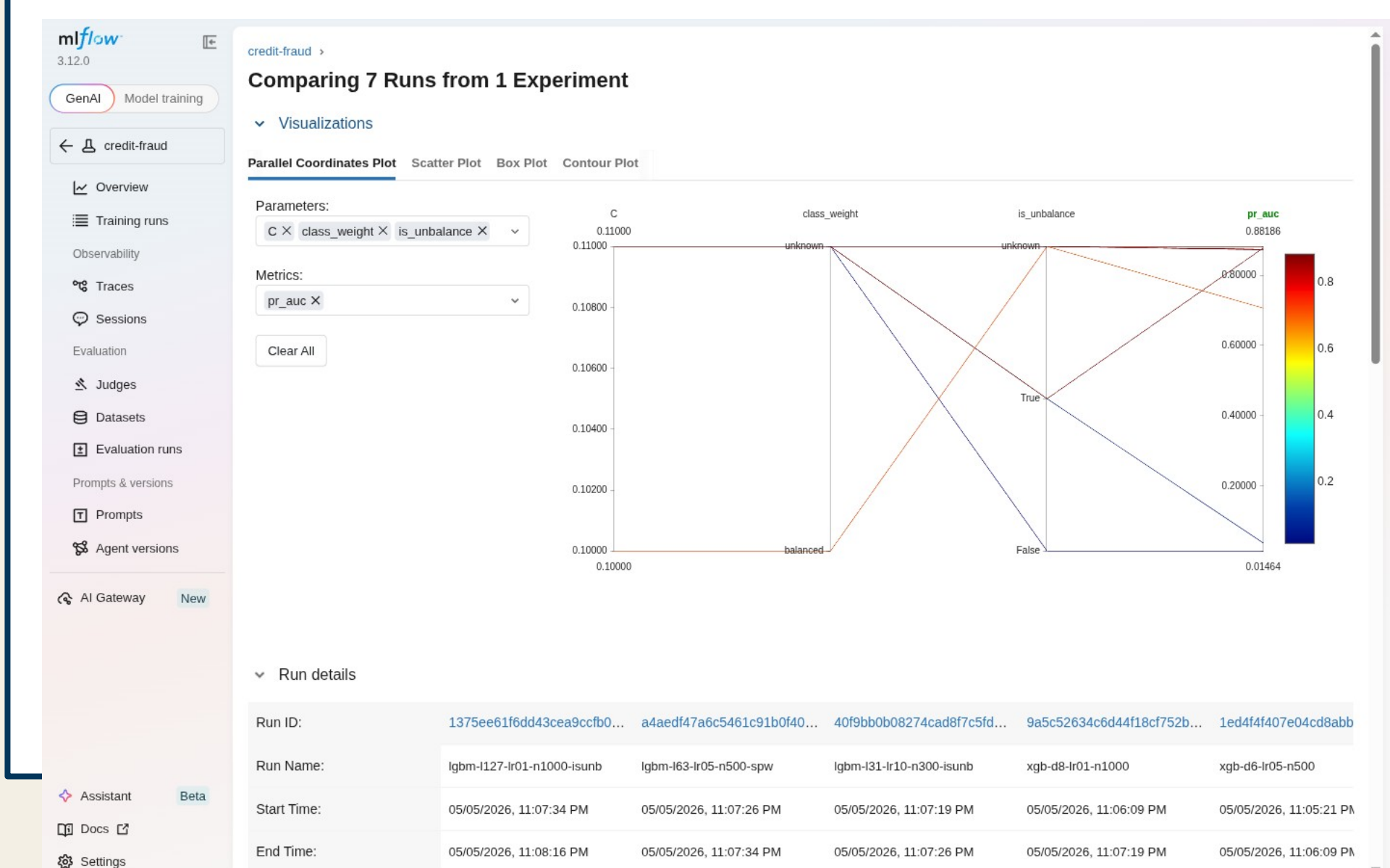
Eduardo García López y Fernando Ramos Ríos

• ITESO — Integración de Servicios de Aprendizaje Automático (ISAA) — Primavera 2026

INTRODUCCIÓN

El fraude con tarjeta de crédito ocurre en menos del 0.2% de las transacciones, lo que vuelve el problema severamente desbalanceado y exige métricas centradas en la clase positiva (PR-AUC en lugar de accuracy). Trabajamos con el conjunto público de la ULB (OpenML 1597, ~284 807 transacciones, 29 atributos numéricos), y atacamos el problema con un pipeline MLOps reproducible.

- Originalmente el proyecto pretendía desplegar en AWS, pero la cuenta académica de uno de los autores fue suspendida sin previo aviso. Reorientamos toda la infraestructura a Cloudflare (R2 como object store, Workers + Containers para inferencia). Esta narrativa de portabilidad — entrenar en Python, exportar a ONNX, servir como contenedor en el edge — es justamente lo que MLOps promete.



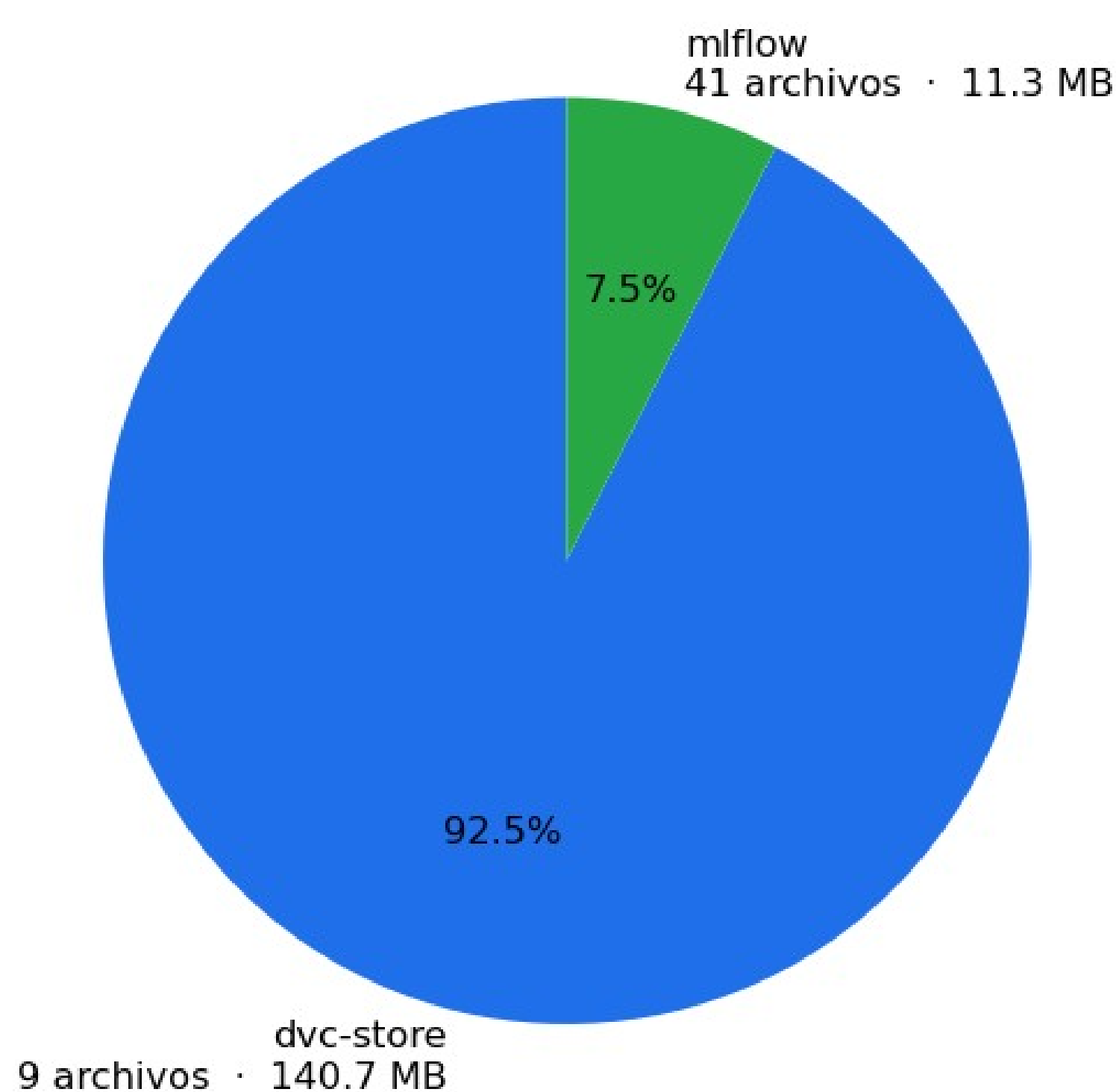
MATERIALES Y MÉTODOS

El experimento confirma que un pipeline MLOps disciplinado (DVC + MLflow + registro por alias + verificación de paridad) permite mover un modelo desde un notebook hasta un endpoint público sin perder reproducibilidad ni precisión.

- El pivote de AWS a Cloudflare fue forzado pero instructivo: ONNX como formato portátil aísla el entrenamiento de la plataforma de inferencia, y los contenedores en el edge (open beta de Cloudflare) ofrecen una alternativa creíble a SageMaker para clases académicas y prototipos.

- Repositorio: github.com/LaloLalo1999/mlops (PR #1).
- Endpoint en vivo: fraud-detector.eduardo-lalo1999.workers.dev

Contenido del bucket R2 «luci-mlops-fraud»
50 objetos · 152.1 MB total

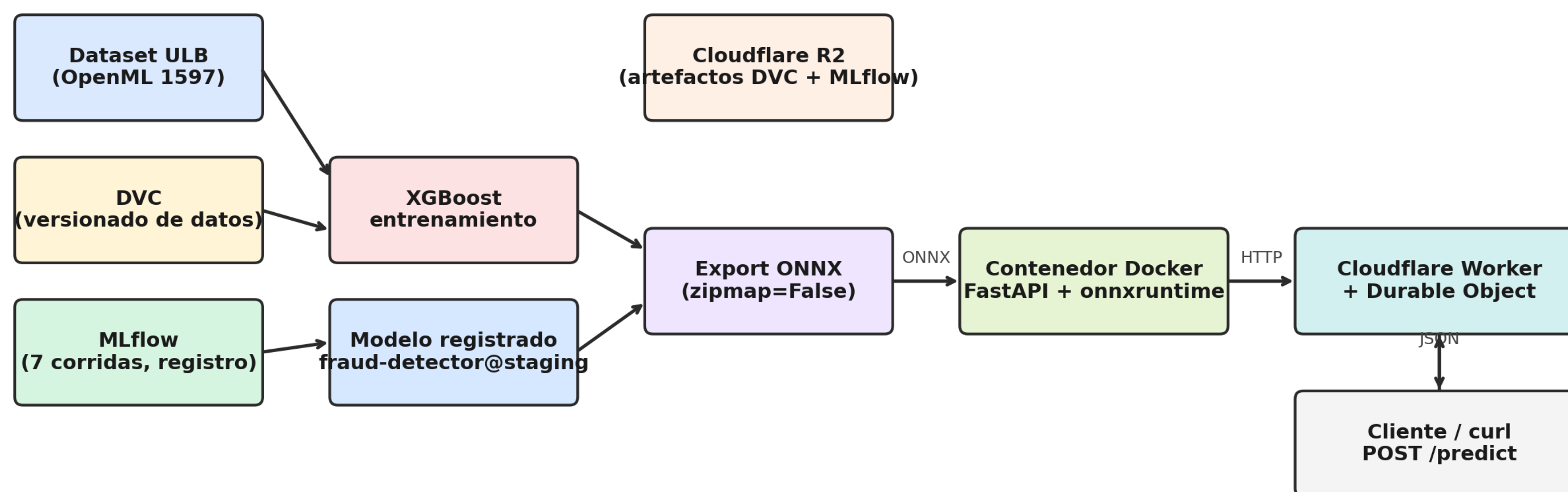


RESULTADOS

Construimos el pipeline en cinco etapas reproducibles, orquestadas por DVC (dvc.yaml + params.yaml) sobre un entorno Python 3.11 fijado por requirements.txt:

- Ingesta y preprocesamiento de OpenML 1597, escalado y partición estratificada 80/20.
- Entrenamiento de 7 corridas con tres familias de modelo (LogReg baseline, LightGBM y XGBoost) variando hiperparámetros y técnicas de balanceo (is_unbalance, scale_pos_weight).
- Registro en MLflow (sqlite:///mlflow.db) — métricas, parámetros, artefactos por corrida y registro del mejor modelo bajo el alias fraud-detector@staging.
- Sincronización de artefactos a Cloudflare R2 (bucket luci-mlops-fraud, 75 objetos, ~167 MB) usando la API de gestión REST, dado que el token cfat_ no firma SigV4.
- Despliegue: exportación a ONNX (opset 15, zipmap=False, paridad verificada a 1e-4 contra sklearn), empaquetado en contenedor Docker (python:3.11-slim + FastAPI + onnxruntime), publicación en el registro de Cloudflare y exposición vía Worker + Durable Object.

Pipeline MLOps — entrenamiento reproducible → inferencia en el edge

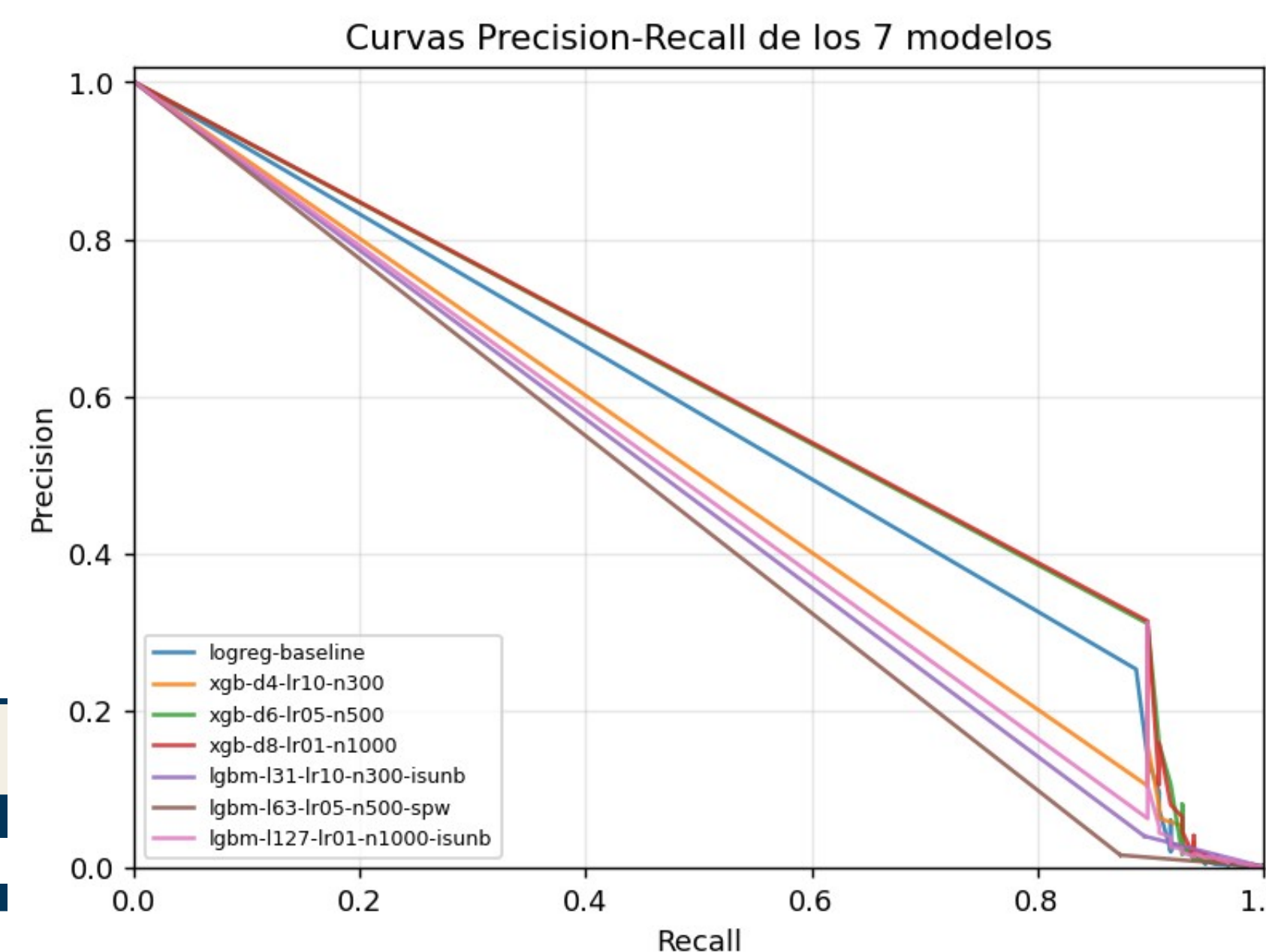
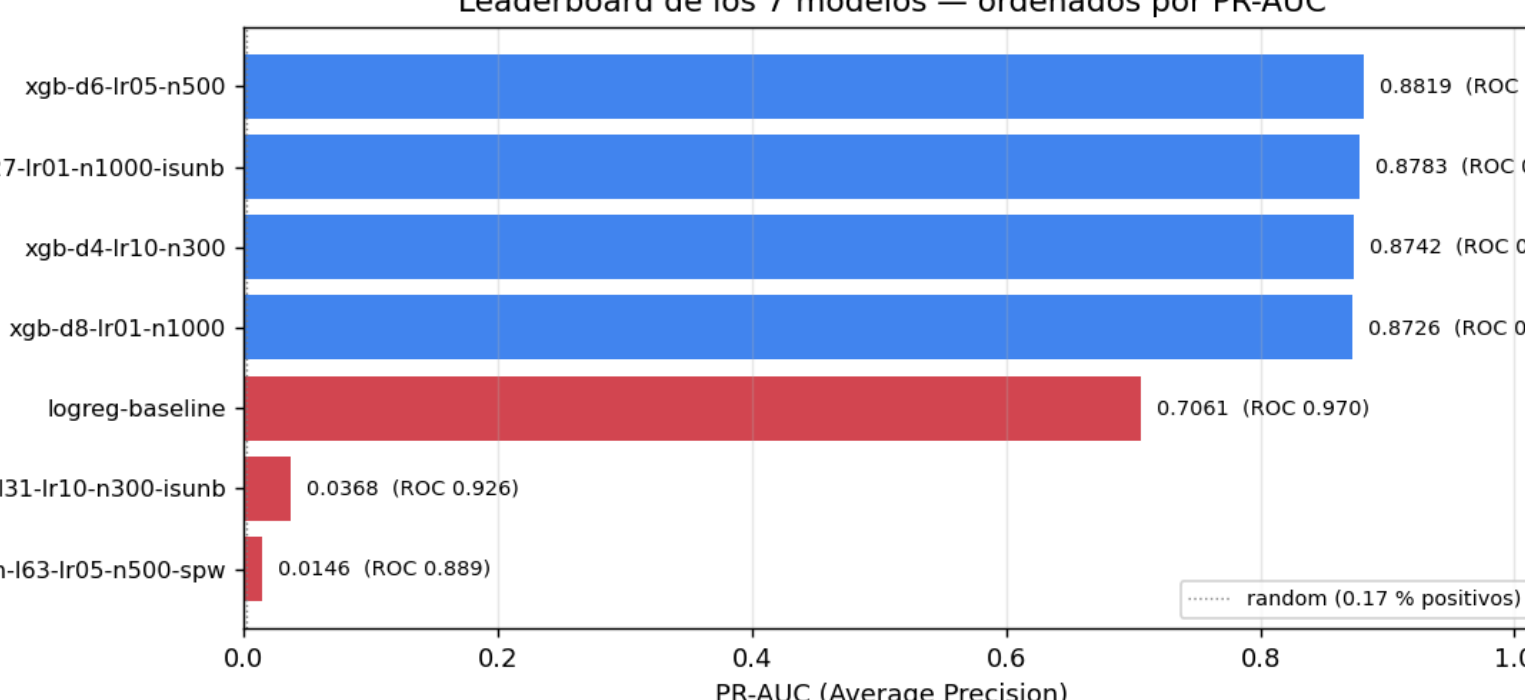


CONCLUSIONES

Mejor modelo: xgb-d6-ir05-n500 con PR-AUC = 0.8819 y ROC-AUC = 0.9789 sobre el conjunto de prueba. Umbral óptimo de decisión: 0.9274, alcanzando F1 = 0.874, precisión = 0.941 y recall = 0.816 — equilibrio adecuado para un detector que se quiere conservador con los falsos positivos.

Dos corridas LightGBM (l31/l63 con balanceo agresivo) colapsaron a PR-AUC ≈ 0.03, mostrando lo fácil que es romper modelos sobre datos desbalanceados — las dejamos visibles como advertencia.

Inferencia en producción: el endpoint POST /predict del contenedor



AGRADECIMIENTOS

A ITESO y al profesor de Integración de Servicios de Aprendizaje Automático por el marco académico y la flexibilidad para validar el despliegue en Cloudflare en lugar de AWS. A nuestros compañeros de ISAA por la retroalimentación durante las tareas previas que alimentaron este proyecto final.